CAD工具包功能模块说明

项目概述

这是一个CAD文件处理工具包，主要用于DXF格式文件的批量处理、块管理、文本更新和自动排版等功能。项目采用Python开发，提供了图形化界面(PyQt5)和命令行两种使用方式。

核心技术栈

- **CAD处理库**: ezdxf - 用于读取、编辑和创建DXF文件
- **数据处理**: pandas - 用于Excel文件的读取和处理
- **图形界面**: PyQt5 - 提供友好的用户交互界面
- **几何计算**: ezdxf.math, ezdxf.bbox - 用于几何对象的计算和边界框分析

功能模块详解

1. CAD文件读取器 (cad_reader.py)

功能描述: 负责读取DXF文件并提取其中的文本对象和几何实体。

核心类: `CADReader`

主要功能:

- 加载DXF文件
- 提取文本对象(TEXT, MTEXT, ATTRIB)
- 提取几何实体(LINE, CIRCLE, ARC, LWPOLYLINE等)
- 递归处理块引用(INSERT)中的嵌套内容
- Unicode转义序列解码(支持\U+XXXX格式)

实现方式:

```
# 使用ezdxf库加载文件
doc = ezdxf.readfile(file_path)
modelspace = doc.modelspace()

# 递归提取文本和几何实体
for entity in modelspace:
    if entity.dxftype() == 'TEXT':
        # 提取文本内容
    elif entity.dxftype() == 'INSERT':
        # 递归处理块引用
```

功能:

- 支持中文Unicode编码的自动解码
- 区分嵌套文本和顶层文本
- 支持多种文本实体类型

---

2. 文本处理器 (text_processor.py)

功能描述: 处理CAD文本内容，生成规范的块名称。

核心类: `TextProcessor`

主要功能:

- 清理DXF格式标记
- 生成块名称(支持多种策略)
- 文本内容规范化

文本策略:

1. `first_valid` : 使用第一个有效文本
2. `combine` : 组合所有文本内容

实现方式:

- 使用正则表达式清理格式标记
- 过滤无效字符和空白
- 组合多个文本片段

---

3. 块创建器 (block_creator.py)

功能描述: 将CAD文件中的图形对象组合成块，并使用文本内容命名。

核心类: `BlockCreator`

主要功能:

- 创建块定义
- 将实体添加到块中
- 替换原始实体为块引用
- 添加块属性(材质、厚度、物料ID等)
- 批量处理CAD文件

实现流程:

1. 读取CAD文件获取文本和几何实体
2. 根据文本内容生成块名
3. 创建新的块定义
4. 将所有实体复制到块中
5. 删除原始实体
6. 在模型空间插入块引用
7. 添加属性信息

实现方式:

```
# 创建块
block = doc.blocks.new(name=block_name)

# 复制实体到块
for entity in entities:
    block_entity = entity.copy()
    block.add_entity(block_entity)

# 插入块引用
blockref = modelspace.add_blockref(block_name, insertion_point)

# 添加属性
blockref.add_attrib(tag='材质', text=material_val)
```

---

4. 块查找器 (block_finder.py)

功能描述: 根据Excel文件中的物料ID或图号，在DXF文件中查找对应的块并合并。

核心类: `BlockFinder`

主要功能:

- 加载Excel文件提取物料信息
- 在DXF文件中搜索匹配的块
- 按材质和厚度分组
- 智能布局和排版
- 合并块到新文件
- 更新Excel文件标记查找结果

Excel列识别:

- 物料ID/物料编号
- 图号/图纸编号
- 总数量
- 材质/材料
- 厚度/板厚
- 名称/零件名称

匹配策略:

1. 直接文本匹配
2. 无空格匹配
3. 规范化模糊匹配(处理连字符不一致)
4. 优先级过滤(物料ID优先于图号)

布局算法:

- 按材质分组
- 按厚度分行
- 支持中心对齐和底部对齐
- 可配置块间距和组间距
- 计算块的边界框

实现方式:

```
# 使用Importer跨文档复制块
importer = Importer(source_doc, new_doc)
importer.import_entities(entities, target_layout=new_block)
importer.finalize()

# 计算块的边界框
from ezdxf import bbox
extents = bbox.extents(block)
width = extents.extmax.x - extents.extmin.x
height = extents.extmax.y - extents.extmin.y
```

5. Excel读取器 (excel_reader.py)

功能描述: 读取Excel文件，提取物料信息，并更新CAD文本。

核心类: `ExcelReader`

主要功能:

- 读取Excel文件
- 识别列名(支持多种别名)
- 根据物料ID或图号更新CAD文本
- 查找匹配的行数据

实现方式:

```
import pandas as pd

# 读取Excel
df = pd.read_excel(excel_file)

# 创建映射
for index, row in df.iterrows():
    material_id = row['物料ID']
    total_qty = row['总数量']
    mapping[material_id] = total_qty
```

---

6. 自动排版器 (auto_nesting.py)

功能描述: 自动排版CAD图形，按材质和厚度分组布局。

核心类:

- `CadItem`: 表示单个CAD图形项
- `ShelfPacker`: 货架式打包算法
- `AutoNester`: 自动排版主类

排版算法:

1. 收集所有图形项
2. 提取材质和厚度信息
3. 按材质和厚度分组

材质厚度识别:

- 从文本中提取"材质: xxx"

- 从文本中提取"厚度: xxx"
- 支持"材质名 "如06Cr19Ni10"

实现方式:

```
# Shelf打包算法
if current_x + item.width > bin_width:
    # 换行
    current_y += shelf_height + spacing
    current_x = 0

item.x = current_x
item.y = current_y
current_x += item.width + spacing
```

---

7. CAD文件合并器 (cad_merge.py)

功能描述: 合并多个DXF文件到一个文件中。

主要功能:

- 批量合并DXF文件
- 使用Importer确保完整导入
- 自动布局排列
- 可选显示文件名标签

实现方式:

```
# 为每个文件创建容器块
container_block = doc.blocks.new(name=f"MERGE_{i}_{filename}")

# 使用Importer导入
importer = Importer(source_doc, doc)
importer.import_entities(src_entities, target_layout=container_block)
importer.finalize()

# 在模型空间插入块引用
msp.add_blockref(container_block_name, (x, y))
```

---

8. 图形用户界面 (cad_toolkit_gui.py)

功能描述: 提供友好的图形化操作界面。

核心类: 多个选项卡类

主要选项卡:

8.1 块批量导出

- 将DXF文件中的所有块导出为单独的文件
- 支持文本处理和块名生成
- 显示导出进度

8.2 块批量创建

- 批量处理CAD文件创建块
- 支持Excel数据关联
- 可配置属性写入
- 支持清理现有块

8.3 块筛寻和合并

- 根据Excel查找块
- 智能布局和分组
- 可配置对齐方式和间距
- 支持重复线删除

8.4 文本内容更改

- 根据Excel更新CAD文本
- 支持数量追加模式
- 支持文本重构模式

8.5 CAD文件合并

- 批量合并DXF文件
- 可配置间距
- 可选显示文件名

8.6 BOM数量计算器

- 处理Excel文件
- 计算总数量
- 验证序号完整性
- 检查父子关系

8.7 自动排版

- 按材质厚度分组排版

线程处理:
所有耗时操作都使用QThread异步执行，避免界面冻结:

```
class ExportBlocksWorker(QThread):
    progress = pyqtSignal(str)
    finished = pyqtSignal(bool, str)

    def run(self):
        # 执行导出操作
        self.export_blocks(...)
```

辅助模块

9. 分析工具 (analyze_dxf.py)

功能: 分析DXF文件结构，检测问题

- 统计块定义数量
- 检测相似块名
- 统计块引用
- 检测断开的引用
- 检测重复的图号

10. 调试工具

多个debug_*.py文件用于调试特定问题:

- `debug_block_search.py`: 调试块搜索
- `debug_excel_extraction.py`: 调试Excel提取
- `debug_text_encoding.py`: 调试文本编码
- `debug_unicode_decoding.py`: 调试Unicode解码

数据流程

典型工作流程1: 块创建

```
CAD文件 → CADReader(读取) → TextProcessor(生成块名)
→ BlockCreator(创建块) → 输出CAD文件
```

典型工作流程2: 块查找合并

```
Excel文件 → 提取物料信息 → 在DXF中搜索块
→ 按材质厚度分组 → 智能布局 → 合并输出
```

典型工作流程3: 自动排版

```
CAD文件/目录 → 收集图形项 → 提取材质厚度
→ 分组 → 输出排版文件
```

关键技术点

1. Unicode处理

支持多种Unicode转义格式:

- `\U+XXXX`
- `\UXXXX`
- `\u+XXXX`
- `\uXXXX`

2. 块引用处理

- 递归处理嵌套块
- 使用Importer确保依赖完整
- 自动解析块变换(缩放、旋转、平移)

3. 边界框计算

- 优先使用ezdxf.bbox
- 回退到手动计算
- 支持多种实体类型

4. 文本匹配

- 直接匹配
- 无空格匹配
- 规范化匹配(统一连字符)
- 优先级过滤

5. 重复检测

基于几何特征哈希:

- 线段: 端点(无序)
- 圆: 圆心+半径
- 圆弧: 圆心+半径+角度
- 多段线: 所有顶点

配置文件

block_resource_list.csv

存储块资源信息的CSV文件

BLOCK_STANDARDS.md

块命名和创建的标准文档

打包和部署

使用PyInstaller打包:

- `CADToolkit.spec`: 主程序打包配置
- `cad_toolkit_gui.spec`: GUI程序打包配置
- `CAD工具包.spec`: 中文名称打包配置

总结

这个CAD工具包是一个DXF文件处理系统，涵盖了:

- **文件读写**: 基于ezdxf的完整DXF操作
- **数据关联**: Excel与CAD的双向关联
- **智能处理**: 自动识别、匹配、分组
- **批量操作**: 支持大规模文件批处理
- **用户友好**: 图形界面和进度反馈